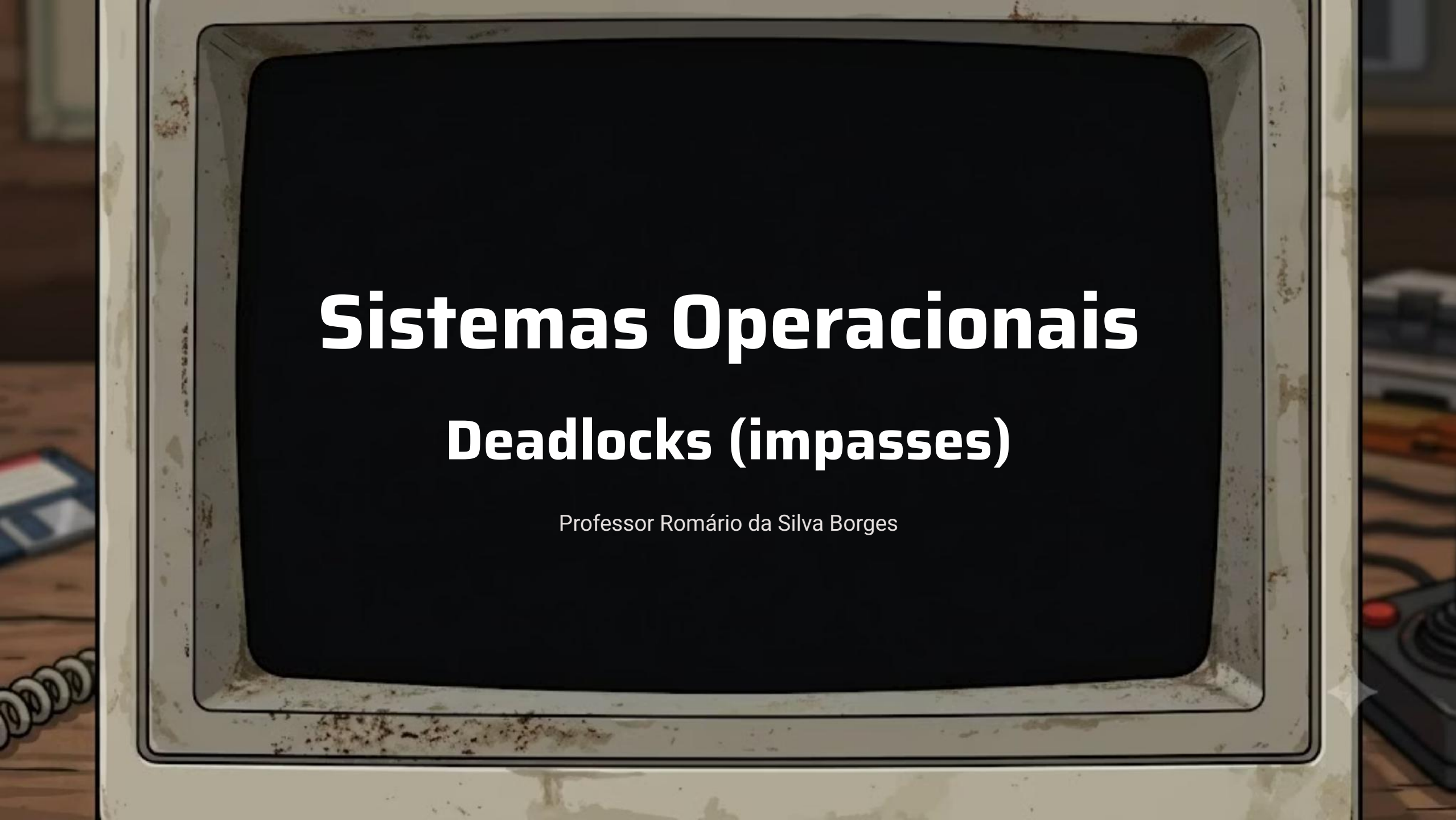




Sistemas Operacionais

Deadlocks (impasses)

Professor Romário da Silva Borges

O que é deadlock?



Introdução

O controle de concorrência entre tarefas acessando recursos compartilhados implica em suspender algumas tarefas enquanto outras acessam os recursos, de forma a garantir a consistência dos mesmos. Para isso, a cada recurso é associado um semáforo ou outro mecanismo equivalente. Assim, as tarefas solicitam e aguardam a liberação de cada semáforo para poder acessar o recurso correspondente.

Em alguns casos, o uso de semáforos ou mutexes pode levar a situações de impasse (ou deadlock), nas quais todas as tarefas envolvidas ficam bloqueadas aguardando a liberação de semáforos, e nada mais acontece.

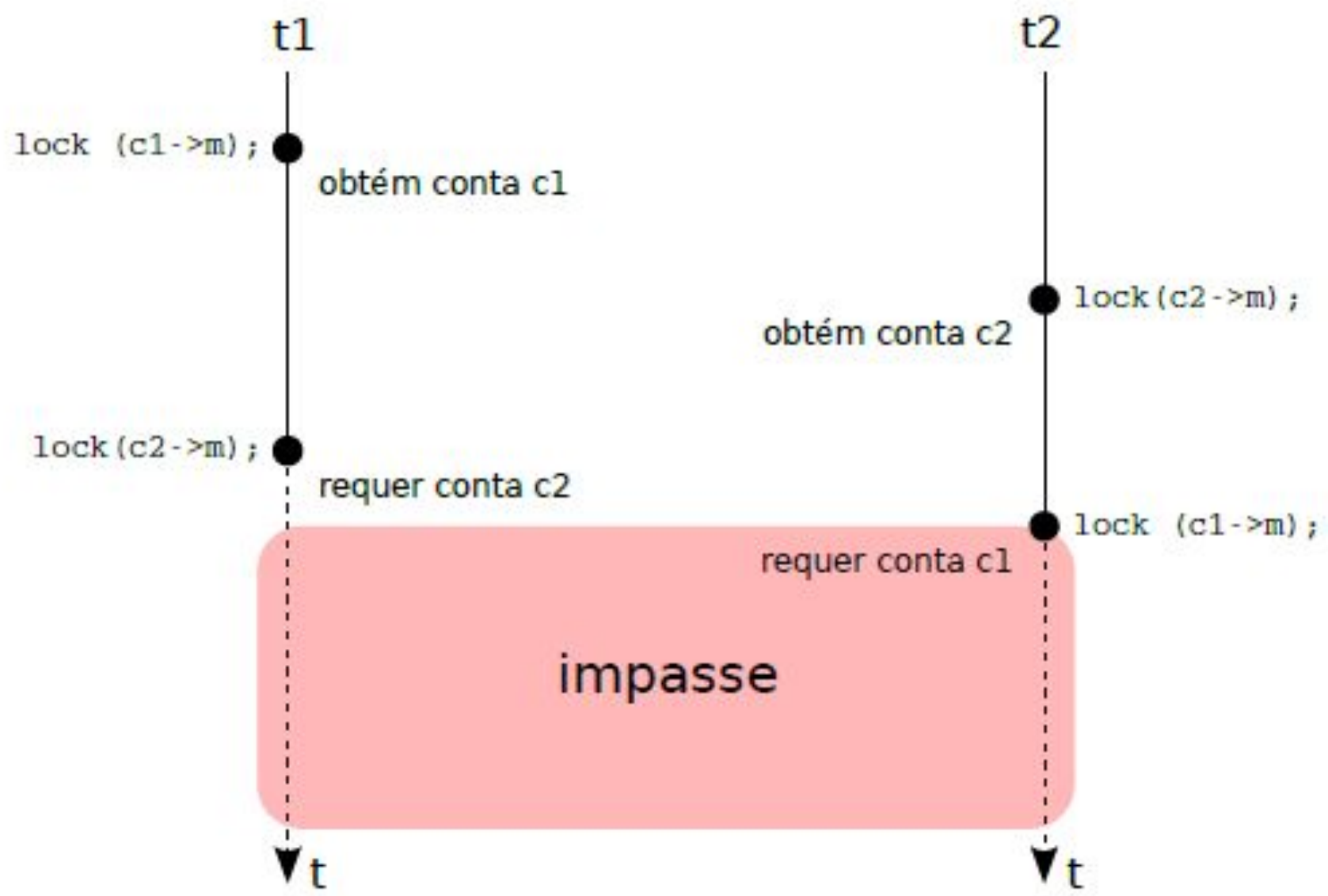


Figura 13.1: Impasse entre duas transferências.

```

1 typedef struct conta_t
2 {
3     int saldo ;           // saldo atual da conta
4     mutex m ;             // mutex associado à conta
5     ...                   // outras informações da conta
6 } conta_t ;
7
8 void transferir (conta_t* contaDeb, conta_t* contaCred, int valor)
9 {
10     lock (contaDeb->m) ;   // obtém acesso a contaDeb
11     lock (contaCred->m) ;  // obtém acesso a contaCred
12
13     if (contaDeb->saldo >= valor)
14     {
15         contaDeb->saldo -= valor ; // debita valor de contaDeb
16         contaCred->saldo += valor ; // credita valor em contaCred
17     }
18     unlock (contaDeb->m) ;   // libera acesso a contaDeb
19     unlock (contaCred->m) ;  // libera acesso a contaCred
20 }

```

Condições para Deadlock

Condições para deadlock

As **quatro condições de Coffman** devem estar presentes simultaneamente:



Exclusão mútua



Posse e espera

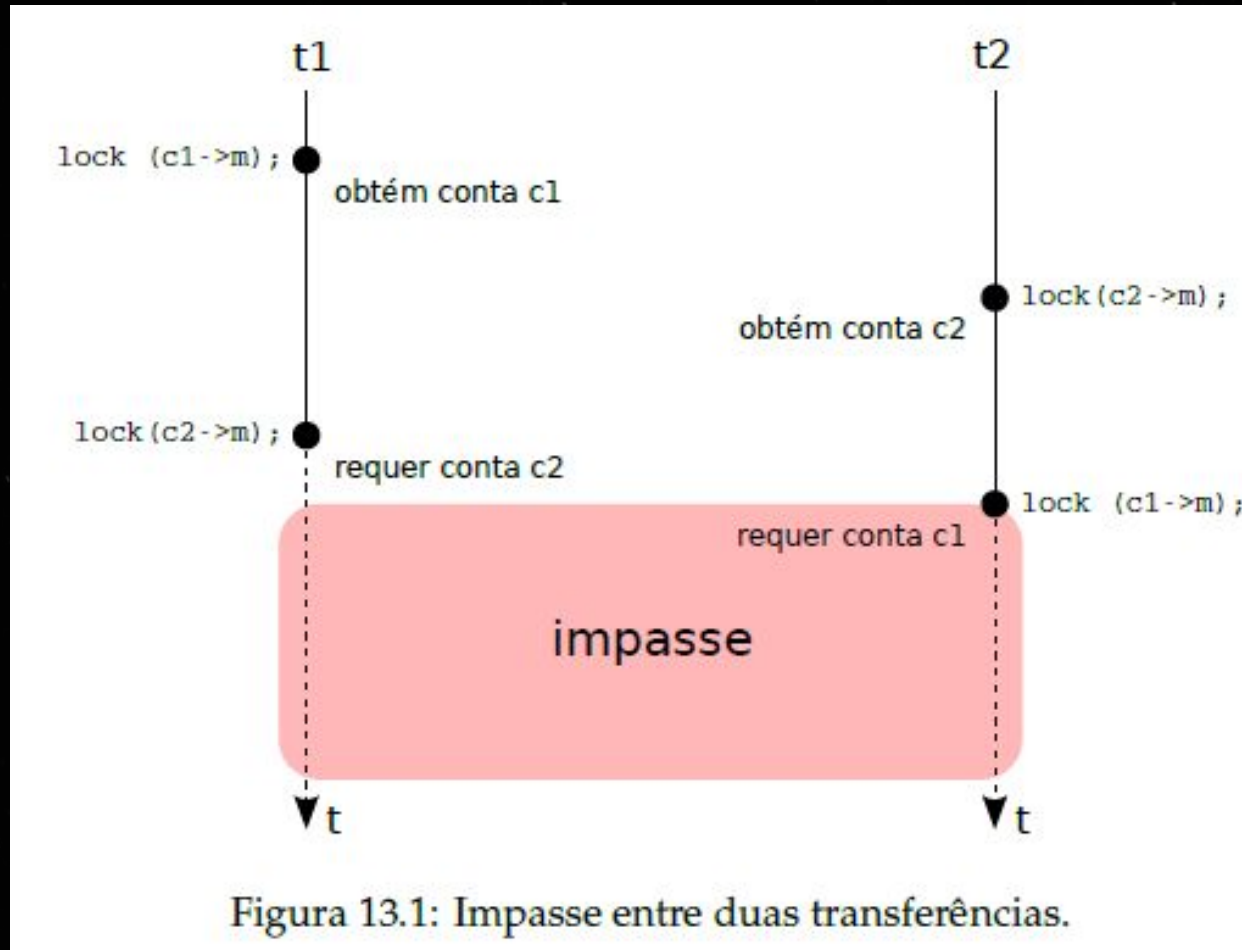


Não preempção



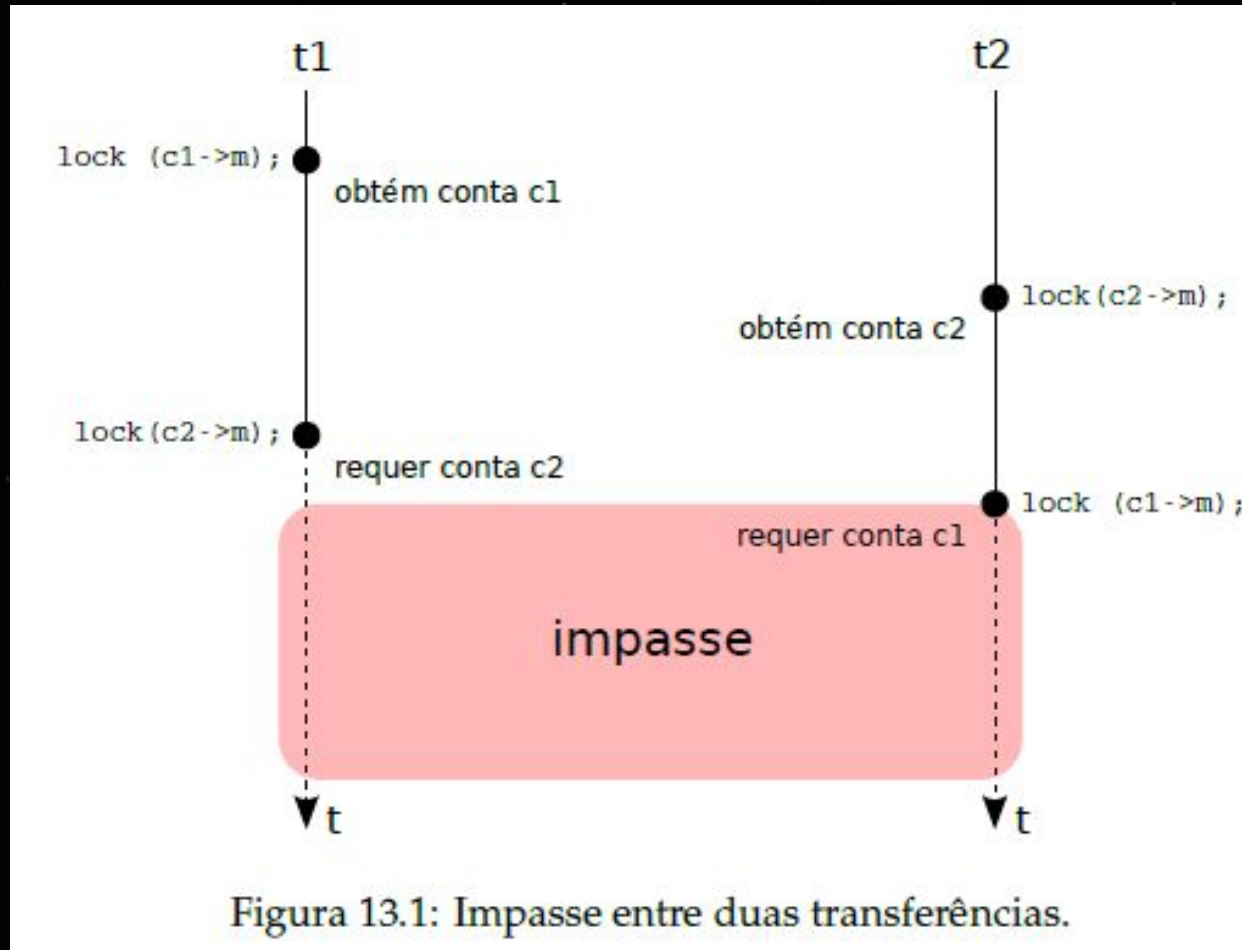
Espera circular

Exclusão mútua



Neste exemplo, apenas uma tarefa por vez pode acessar cada conta.

Posse e Espera



Neste exemplo, cada tarefa obtém o semáforo de uma conta e solicita o semáforo da outra conta para poder prosseguir.

```

typedef int semaphore;
    semaphore resource_1;
    semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

```

(a)

```

semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}

```

(b)

```
typedef int semaphore;
```

```
semaphore resource_1;  
semaphore resource_2;
```

```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

```
void process_B(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

(a)

```
semaphore resource_1;  
semaphore resource_2;
```

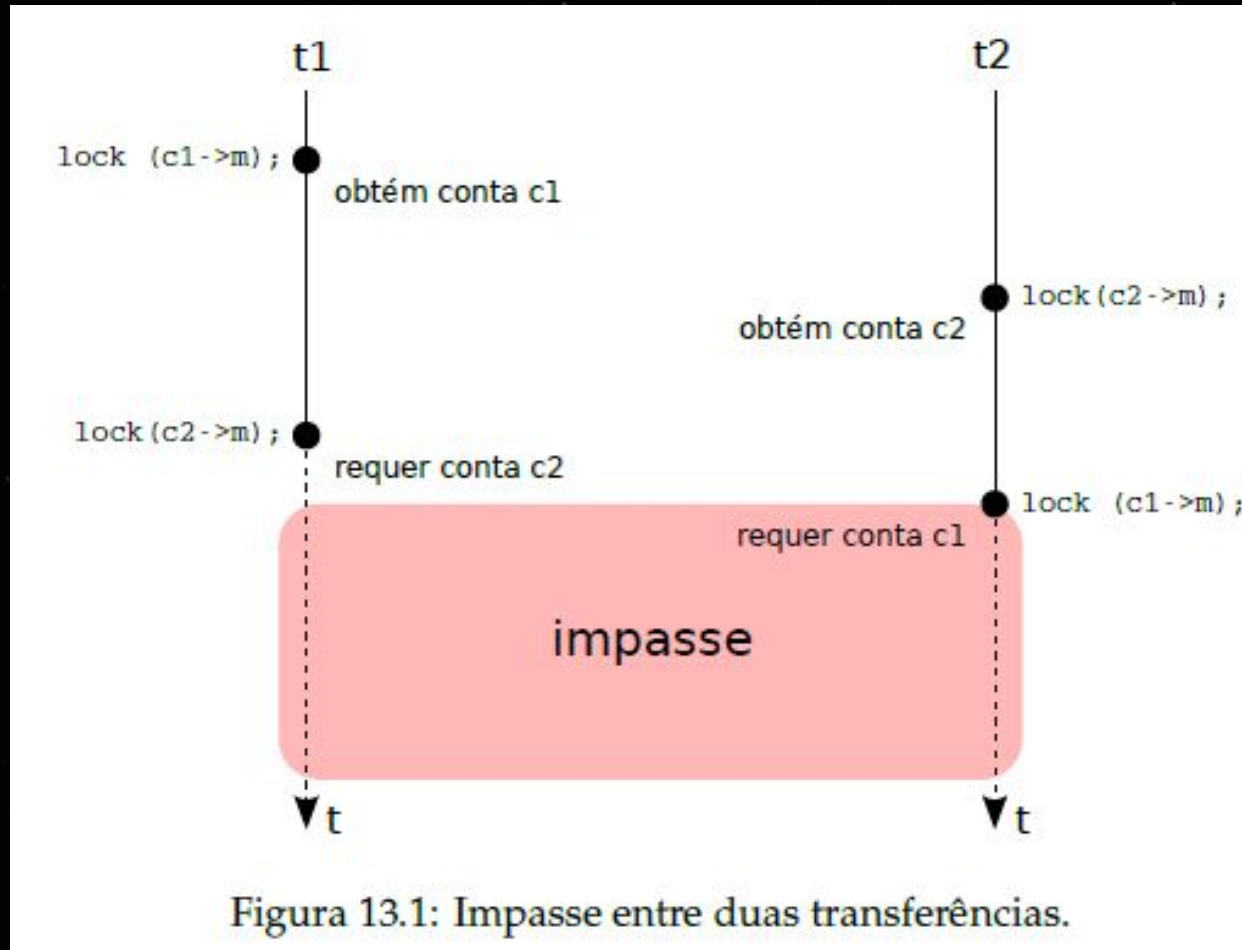
```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

```
void process_B(void) {  
    down(&resource_2);  
    down(&resource_1);  
    use_both_resources( );  
    up(&resource_1);  
    up(&resource_2);  
}
```

(b)

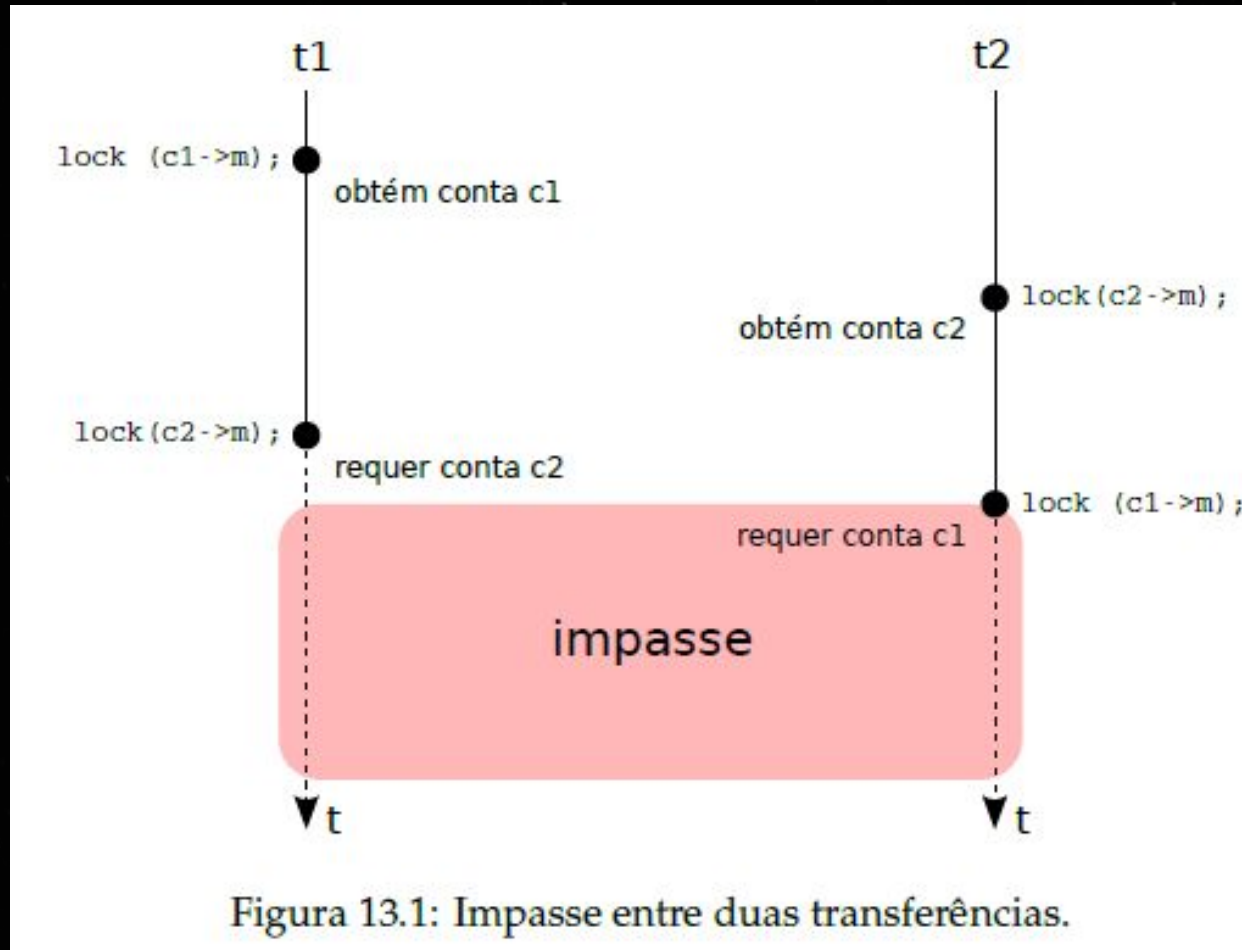
Deadlock em B!

Não-preempção

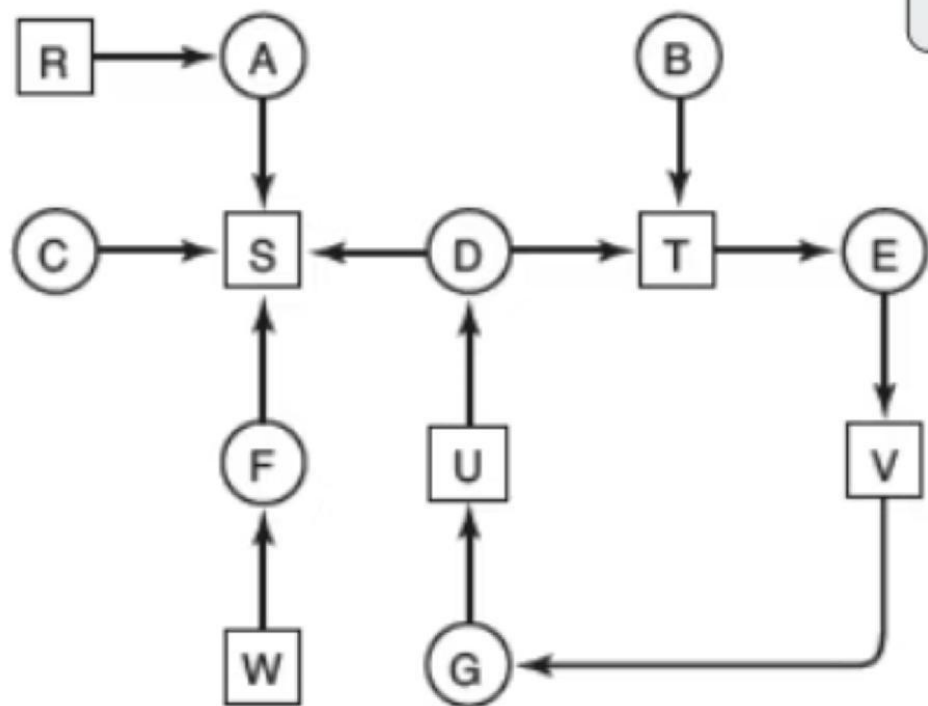


Neste exemplo, cada tarefa detém os mutexes obtidos até liberá-los explicitamente

Não-preempção

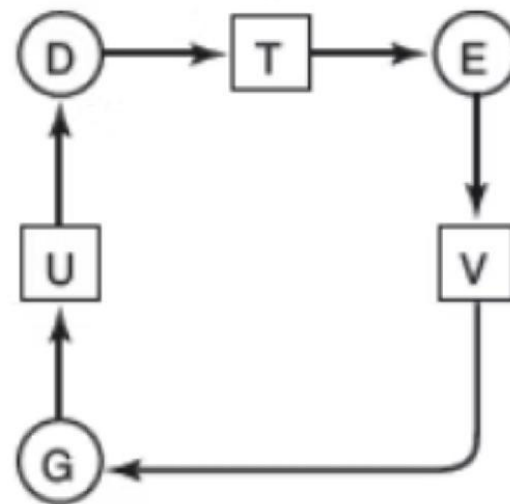


Neste exemplo, pode-se observar claramente que $t_1 \rightarrow t_2 \rightarrow t_1$.



(a)

Deadlock!!!



(b)

Pensando no problema do Jantar dos filósofos:

Condição	Significado	Exemplo no jantar	
1 Exclusão mútua	o recurso só pode ser usado por um processo	o garfo só pode ser segurado por um filósofo	
2 Posse e espera	um processo segura um recurso e espera outro	o filósofo segura o garfo da esquerda e espera o da direita	
3 Não preempção	o recurso não pode ser retirado à força	o filósofo não solta o garfo	
4 Espera circular	há um ciclo de processos esperando entre si	cada filósofo espera o garfo do vizinho	

Atenção!!!

Deve-se observar que essas quatro condições são **necessárias** para a formação de impasses; Se uma delas não for verificada, não existem impasses no sistema.

Por outro lado, **não são condições suficientes** para a existência de impasses, ou seja, a verificação dessas quatro condições não garante a presença de um impasse no sistema.

Essas condições somente são suficientes se existir apenas uma instância de cada tipo de recurso, como será discutido na próxima seção.

Grafos de alocação de recursos

Modelagem com grafos

Grafos de alocação de recursos ajudam a **visualizar e identificar** deadlocks.

Processos

Representados como nós circulares.

Recursos

Representados como nós retangulares.

→ Aresta de posse

Recurso alocado ao processo.

- -> Aresta de requisição

Processo solicitando o recurso.

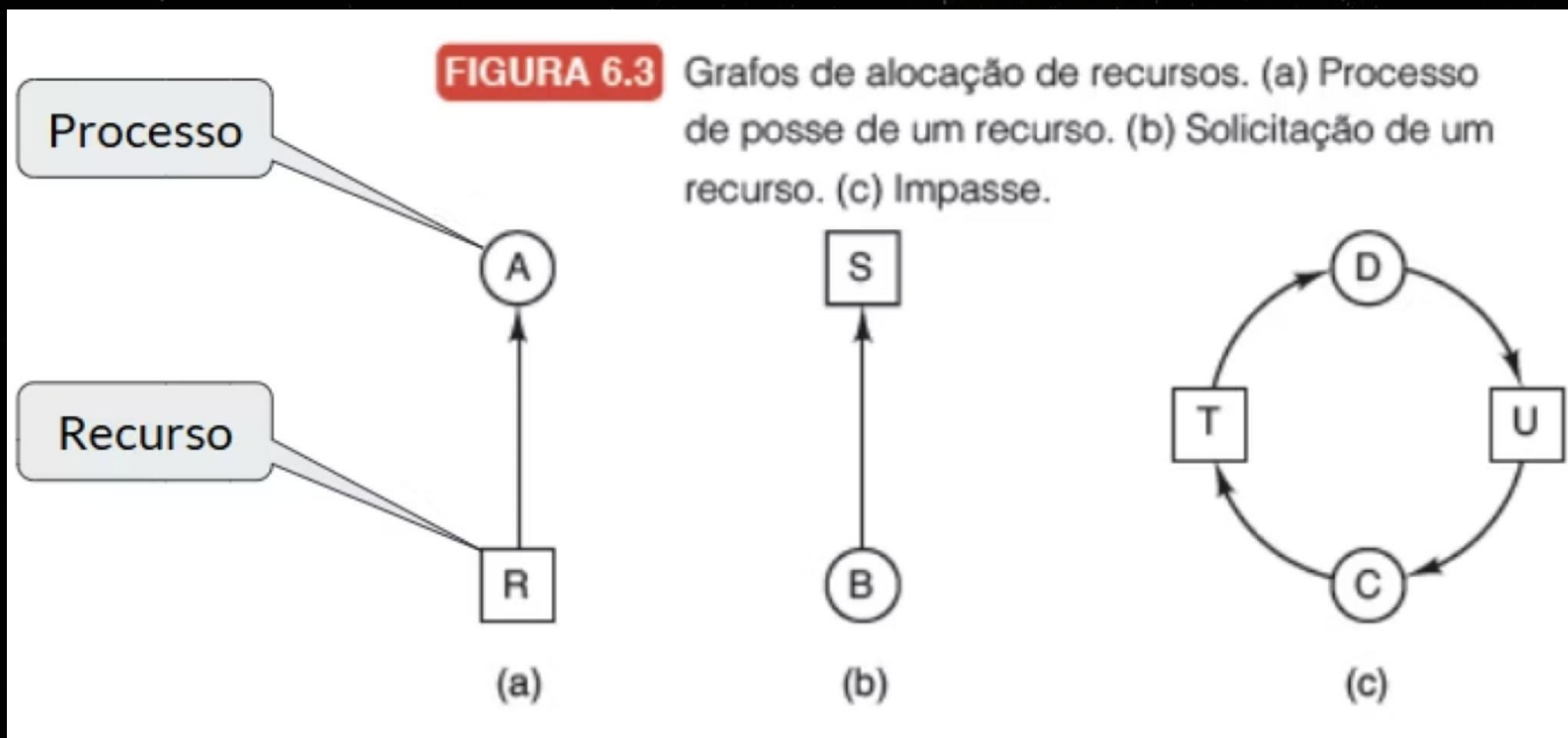
Condições para deadlocks:

1 **Processos no ciclo estão bloqueados**

2 **Aguardam recursos uns dos outros**

3 **Não conseguem prosseguir sozinhos**

Ciclos no grafo



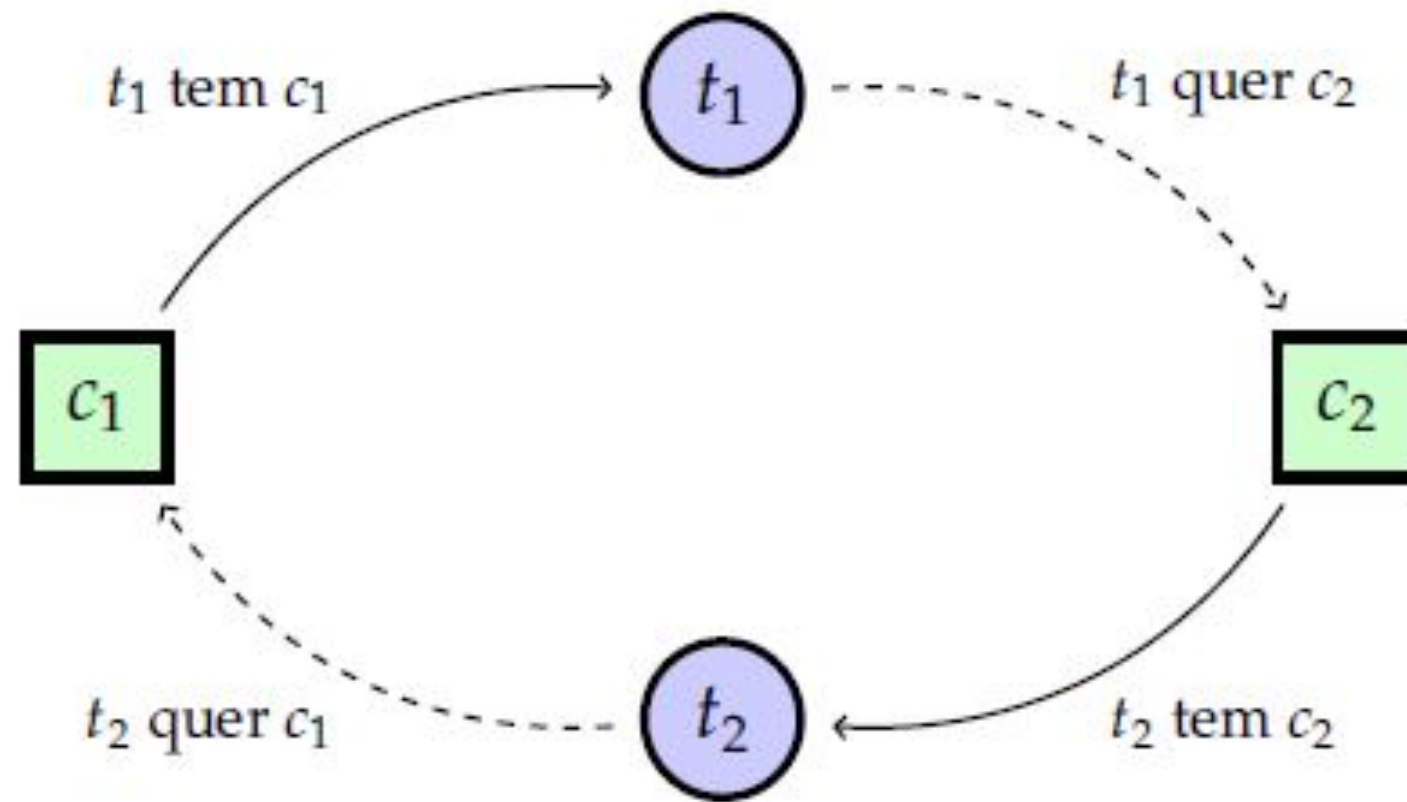


Figura 13.3: Grafo de alocação de recursos com impasse.

$t_1 \dashrightarrow c_2 \rightarrow t_2 \dashrightarrow c_1 \rightarrow t_1$

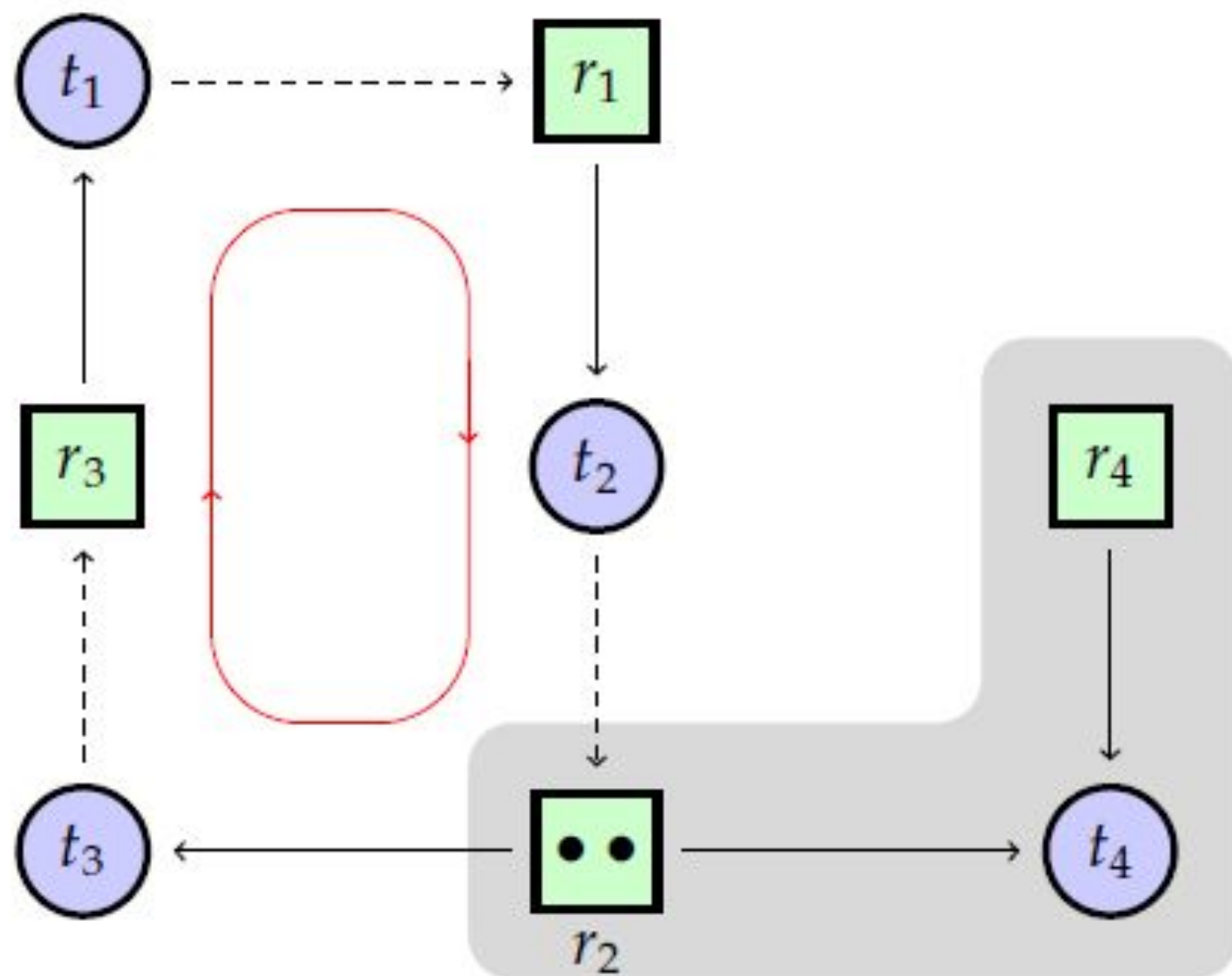


Figura 13.4: Grafo de alocação com múltiplas instâncias de recursos.

Estratégias de Tratamento

Estratégias de tratamento

Formas de lidar com a ocorrência de deadlocks:



Ignorar



Prevenção



Impedimento



Detecção e recuperação de Impasses

1 - Algoritmo do avestruz

Quando a **probabilidade** de ocorrer, a **frequência** e o **impacto** são baixos. A abordagem mais simples é o **algoritmo do avestruz**: "**Fingir que o problema não existe.**"

Como funciona

- Ignora completamente o deadlock
- Aceita o risco de ocorrência
- Evita o custo de tratamento

Quando pode ser aceitável

- Deadlocks são raros
- Custo de prevenção é alto
- Sistema não é crítico



Quando ignorar não é aceitável?

Em **sistemas críticos**, ignorar um deadlock pode causar consequências graves:



Sistemas médicos



Sistemas militares



Sistemas que não podem parar



Alto impacto em caso de falha

2 - Prevenção

Prevenir significa impedir que uma das quatro condições necessárias ocorra. Basta quebrar uma delas:

1

Exclusão mútua

2

Posse e espera

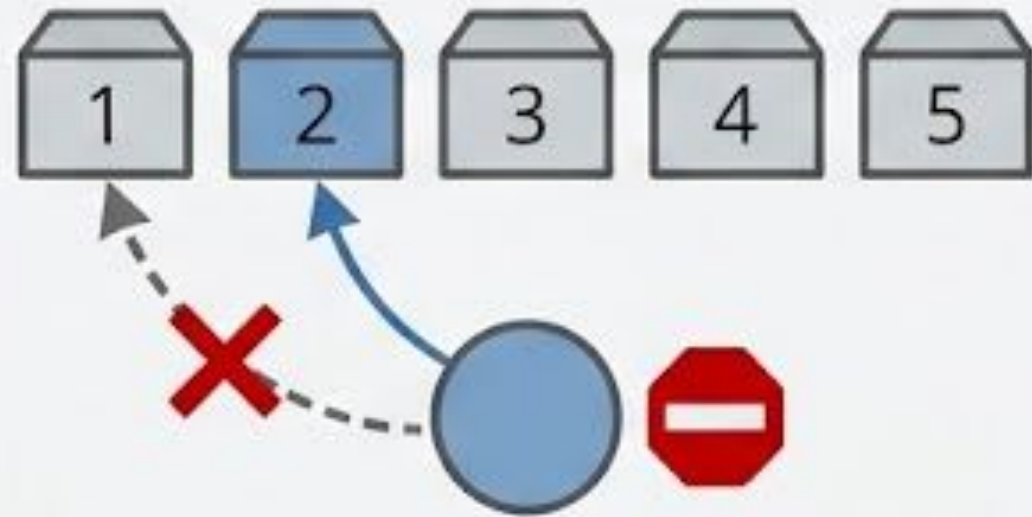
3

Não preempção

4

Espera circular

Prevenção



Prevenção

Prevenção - exclusão mútua

Como garantir a integridade de recursos compartilhados sem usar mecanismos de exclusão mútua?

Uma solução interessante é usada na gerência de impressoras: um processo **servidor de impressão** (printer spooler) gerencia a impressora e atende as solicitações dos demais processos. Com isso, os processos que desejam usar a impressora não precisam obter acesso exclusivo a ela. A técnica de spooling previne impasses envolvendo as impressoras, mas não é facilmente aplicável a certos tipos de recurso, como arquivos em disco e áreas de memória compartilhada.

Prevenção - exclusão mútua

Processo Sem Spooling: Impasse



Dois processos tentam acesso exclusivo.
Impasse ocorre.

Processo Com Spooling (*Printer Spooler*): Solução



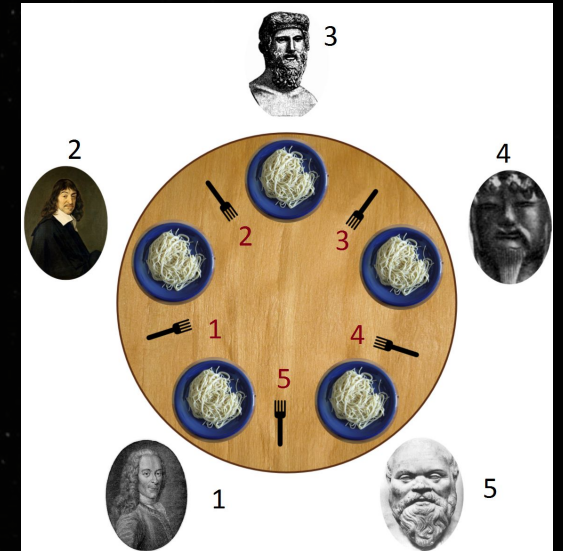
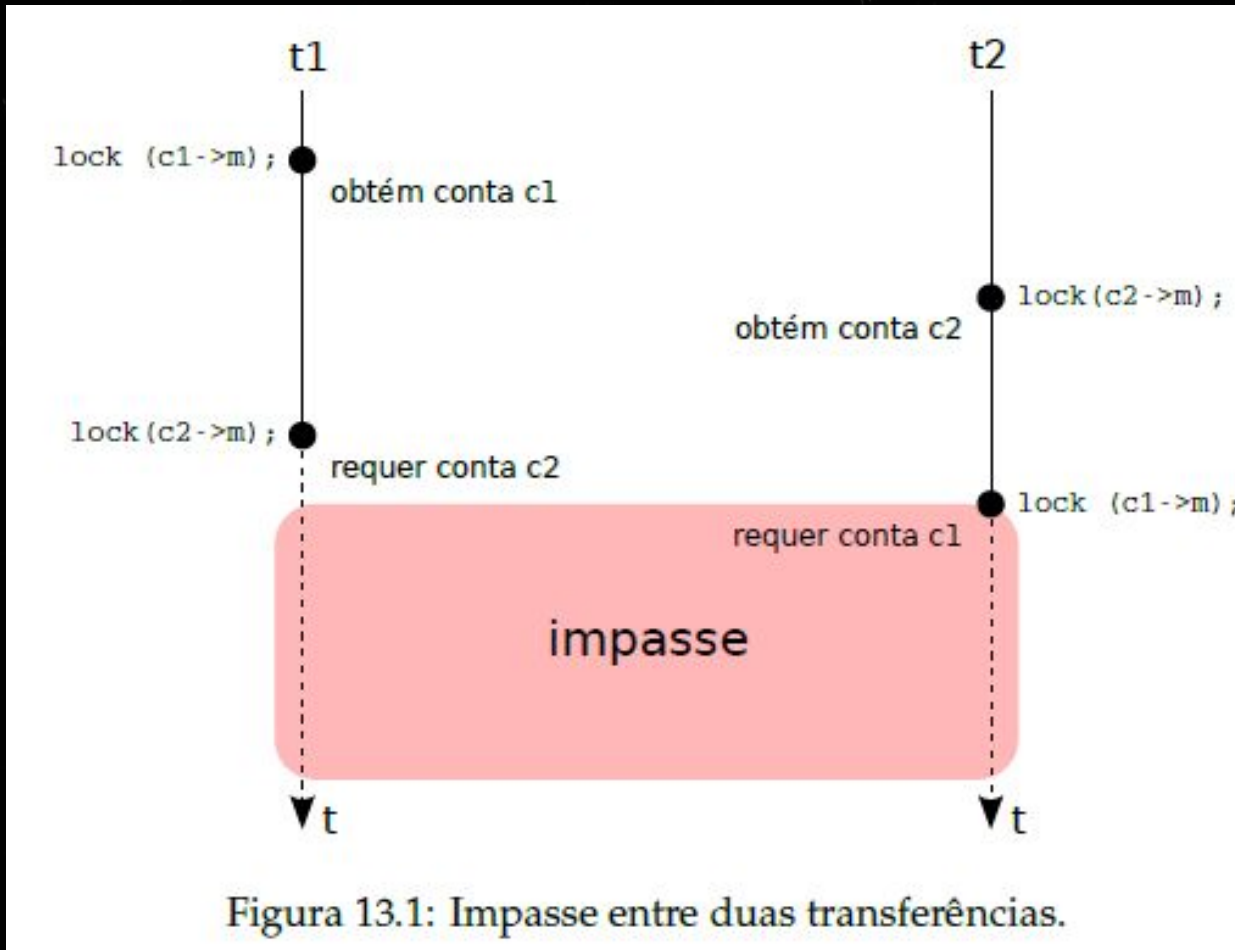
Processos enviam tarefas para o Spooler, sem
acesso exclusivo direto. Impasse prevenido.

Prevenção - Posse e Espera

caso as tarefas usem apenas um recurso de cada vez, solicitando-o e liberando-o logo após o uso, impasses não poderão ocorrer. No exemplo da transferência de fundos, seria possível separar a operação de transferência em duas operações isoladas: débito em c1 e crédito em c2 (ou vice-versa), sem a necessidade de acesso exclusivo simultâneo às duas contas. Com isso, a condição de posse e espera seria quebrada e o impasse evitado.

Uma outra possibilidade seria associar um prazo (time-out) às solicitações de recursos: ao solicitar um recurso, a tarefa define um tempo máximo de espera por ele; caso o prazo expire, a tarefa pode tentar novamente ou desistir, liberando os demais recursos que detém.

Prevenção - Posse e Espera



Prevenção - Posse e Espera

Solução com Time-out: Espera Limitada



Processo define um prazo máximo para obter o recurso solicitado.

Prevenção - Não-preempção

Normalmente uma tarefa obtém e libera os recursos de que necessita, de acordo com sua lógica interna. Se for possível "arrancar" um recurso da tarefa, sem que esta o libere explicitamente, impasses envolvendo aquele recurso não poderão ocorrer. Essa técnica é frequentemente usada em recursos cujo estado interno pode ser salvo e restaurado de forma transparente para a tarefa, como páginas de memória e o próprio processador (nas trocas de contexto). No entanto, é de difícil aplicação sobre recursos como arquivos ou áreas de memória compartilhada, porque a preempção viola a exclusão mútua e pode provocar inconsistências no estado interno do recurso.

Prevenção - Não-preempção

Preempção de Recurso: Solução (*Preemptive*)



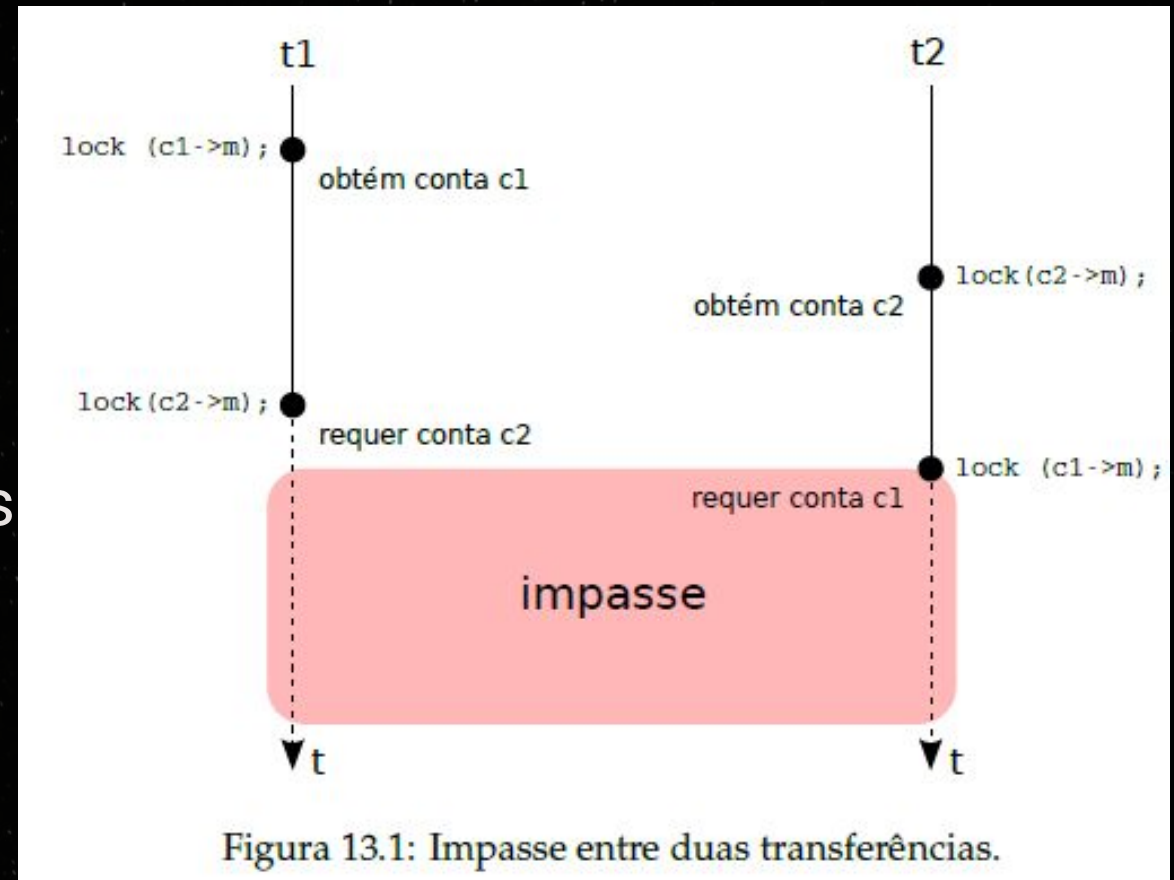
SO "arranca" recurso. Estado é salvo e restaurado de forma transparente.

Prevenção - Espera circular

Um impasse é uma cadeia de dependências entre tarefas e recursos que forma um ciclo. Ao prevenir a formação de tais ciclos, impasses não poderão ocorrer. A estratégia mais simples para prevenir a formação de ciclos é ordenar todos os recursos do sistema de acordo com uma ordem global única, e forçar as tarefas a solicitar os recursos obedecendo a essa ordem.

Prevenção - Espera circular

Neste exemplo, o número de conta bancária poderia definir uma ordem global ($c1 < c2$, por exemplo). Assim, todas as tarefas deveriam solicitar primeiro o acesso à conta mais antiga e depois à mais recente (ou vice-versa, mas sempre na mesma ordem para todas as tarefas). Com isso, elimina-se a possibilidade de impasses.



Prevenção - Espera circular

- para 5 filósofos, os filósofos $f_i=0\dots4$ pegarão o palito p_i e depois o palito p_{i+1} ;
- por sua vez, f_4 deverá pegar o palito p_0 e depois o palito p_4 .

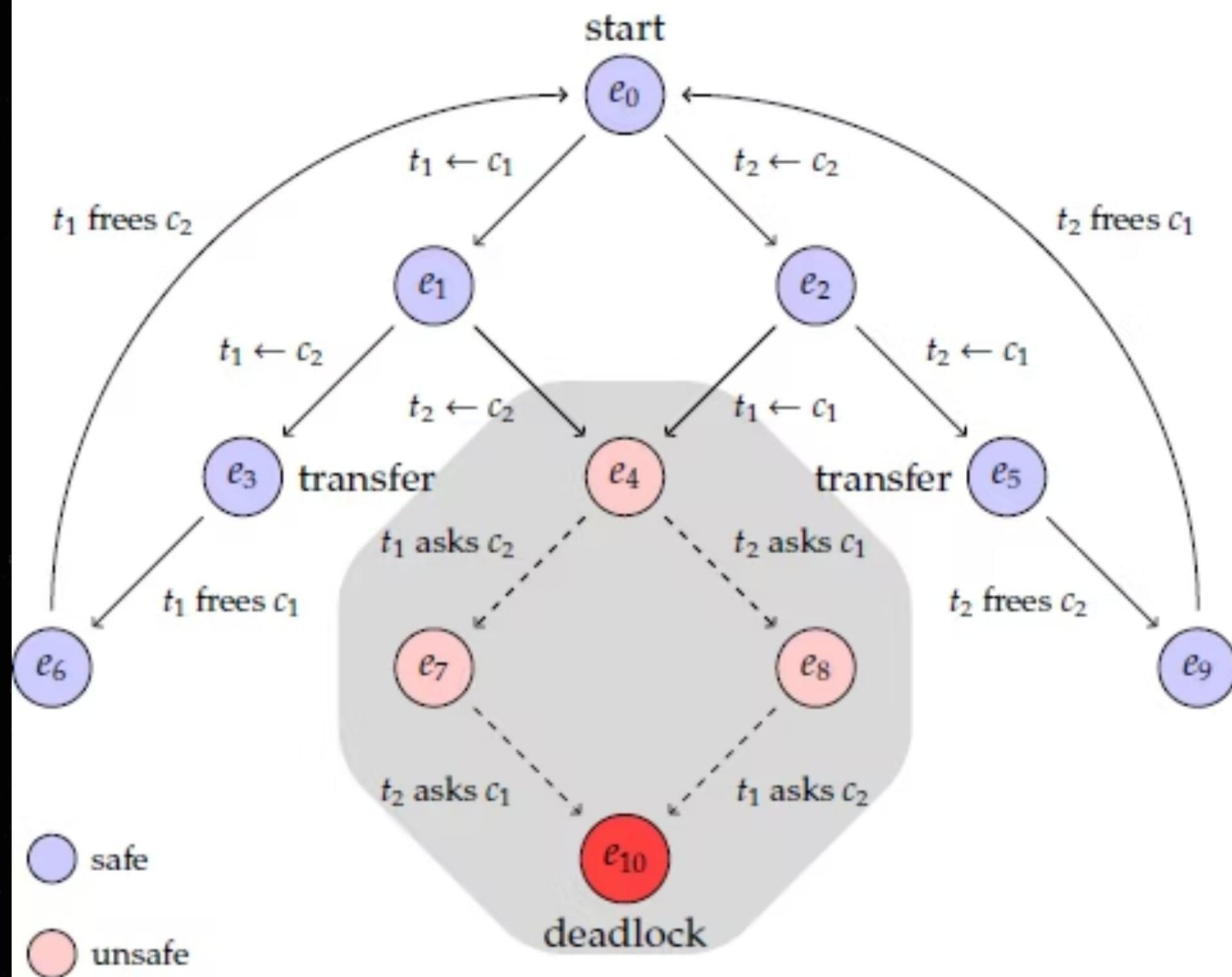


3 - Impedimento

Outra forma de tratar os impasses preventivamente consiste em acompanhar a alocação dos recursos às tarefas e, de acordo com algum algoritmo, negar acessos de recursos que possam levar a impasses. Uma noção essencial nas técnicas de impedimento de impasses é o conceito de **estado seguro**. Cada estado do sistema é definido pela distribuição dos recursos entre as tarefas.

Criado por Dijkstra em 1965, o **algoritmo do banqueiro** faz uma analogia entre as tarefas de um sistema e os clientes de um banco, tratando os recursos como créditos emprestados às tarefas para a realização de suas atividades.





4 - Detecção e resolução de impasses

Resumo: nessa abordagem, o sistema não previne impasses; ele permite que ocorram e depois cuida de identificá-los e resolvê-los. Isso exige responder a duas perguntas:

como detectar e como resolver o impasse?

Detecção e recuperação

A detecção é feita analisando periodicamente o grafo de alocação de recursos em busca de ciclos, que indicam a presença de impasses. O grande desafio está no equilíbrio da frequência dessa análise: verificar com muita frequência tem alto custo computacional e prejudica o desempenho, enquanto verificar raramente atrasa a detecção dos impasses, também prejudicando o sistema.

Depois de detectado o impasse e identificadas as tarefas e recursos envolvidos, o sistema precisa resolvê-lo, o que pode ser feito de diferentes formas.

Eliminar tarefas

uma ou mais tarefas envolvidas no impasse são eliminadas, liberando seus recursos para que as demais tarefas possam prosseguir. A escolha das tarefas a eliminar deve levar em conta vários fatores, como o tempo de vida de cada uma, a quantidade de recursos que cada tarefa detém, o prejuízo para os usuários que dependem dessas tarefas, etc

Eliminar tareas



Retroceder tarefas

uma ou mais tarefas envolvidas no impasse têm sua execução parcialmente desfeita (uma técnica chamada rollback), de forma a fazer o sistema retornar a um estado seguro anterior ao impasse. Para retroceder a execução de uma tarefa, é necessário salvar periodicamente seu estado, de forma a poder recuperar um estado anterior quando necessário. Além disso, operações envolvendo a rede ou interações com o usuário podem ser muito difíceis ou mesmo impossíveis de retroceder: como desfazer o envio de um pacote de rede, ou a reprodução de um arquivo de áudio na tela do usuário?

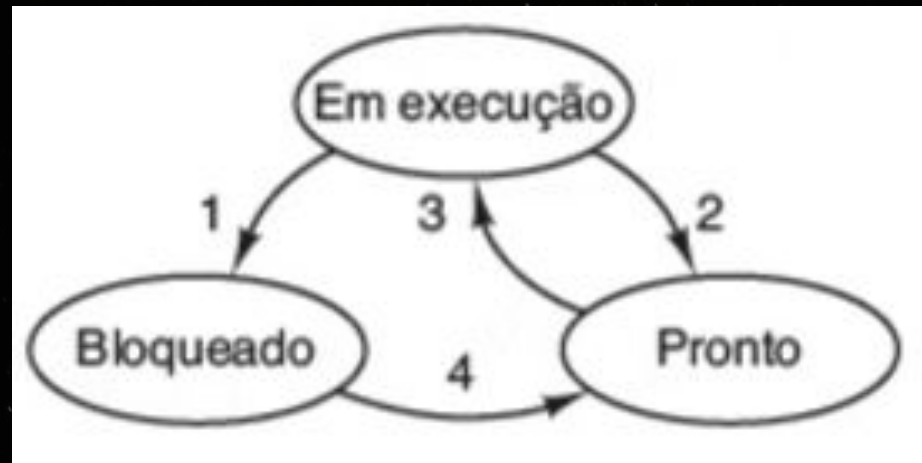
Retroceder tarefas

TRANSAÇÕES EM BANCO DE DADOS

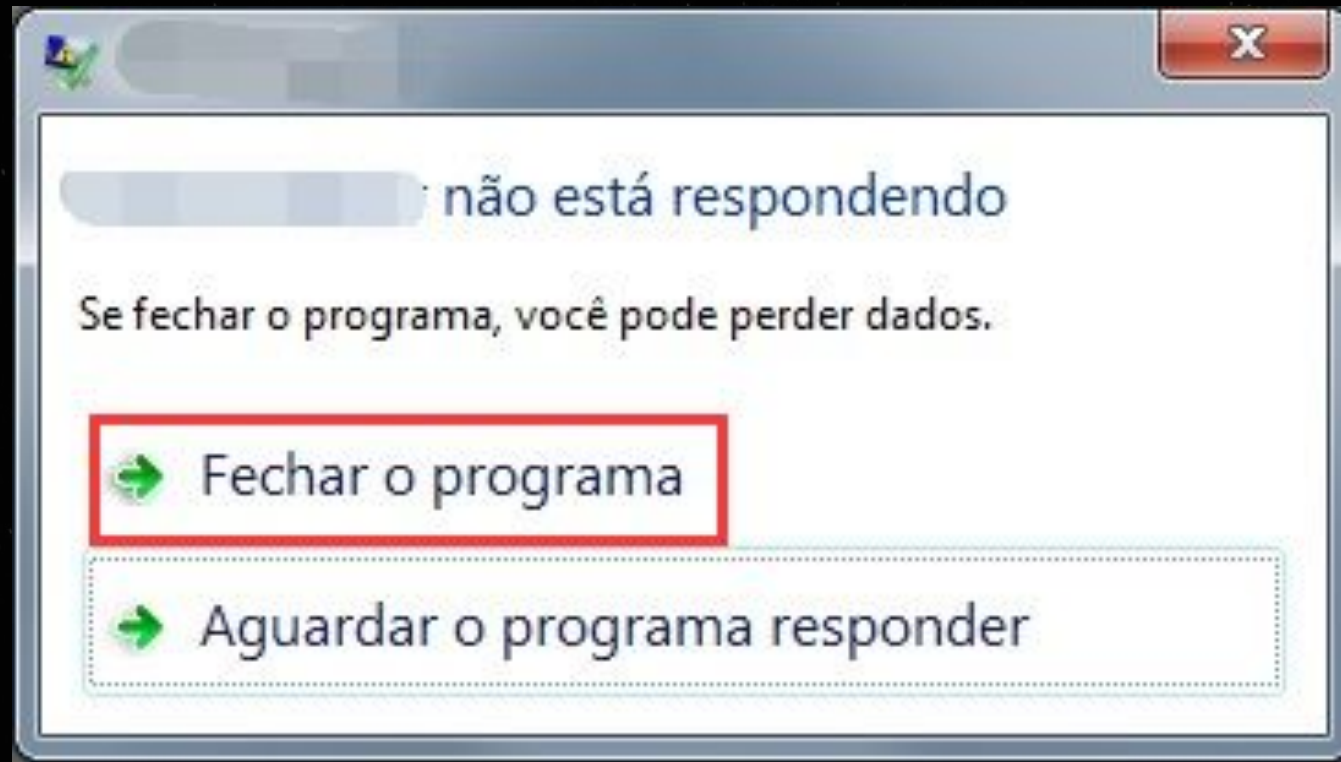


“Em uma situação de Deadlock, o processador está consumindo recurso? o que acontece por trás?”

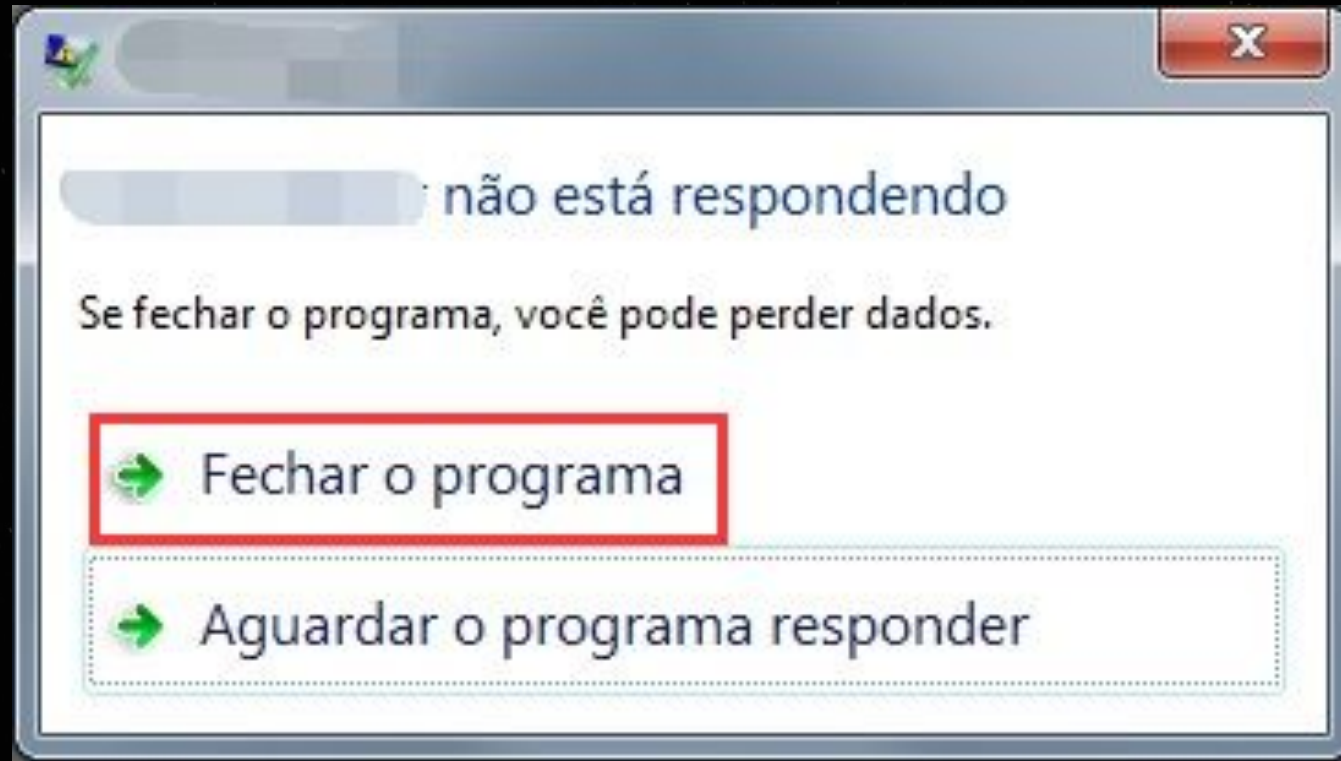
- Aluno BCC 2026



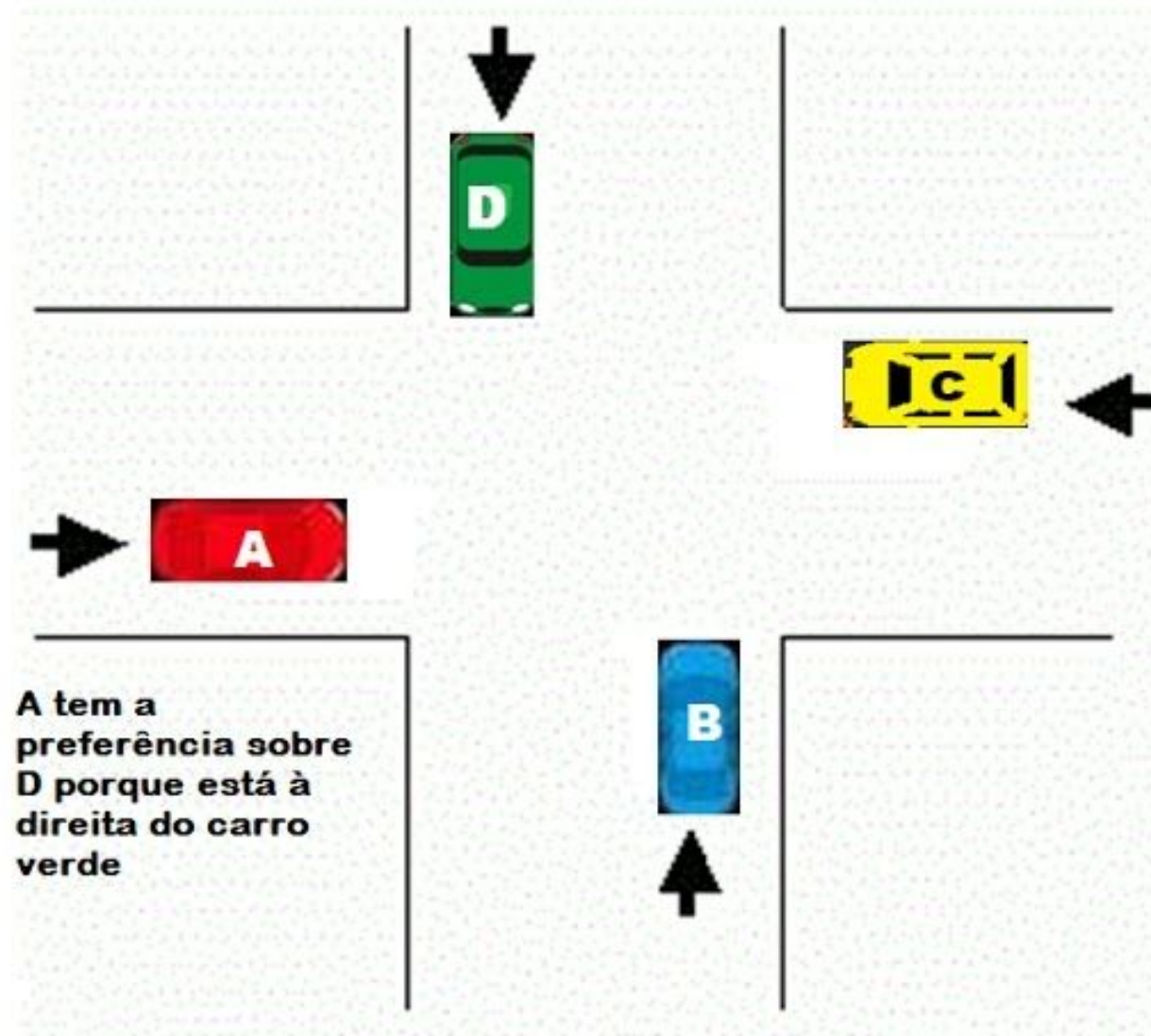
E essa famosa telinha, você acredita que está relacionado a um deadlock?



O Windows detecta que a interface deixou de responder às mensagens dentro de um timeout; isso, por si só, não demonstra que existe um deadlock - mas pode ser uma causa.



Deadlock???



Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

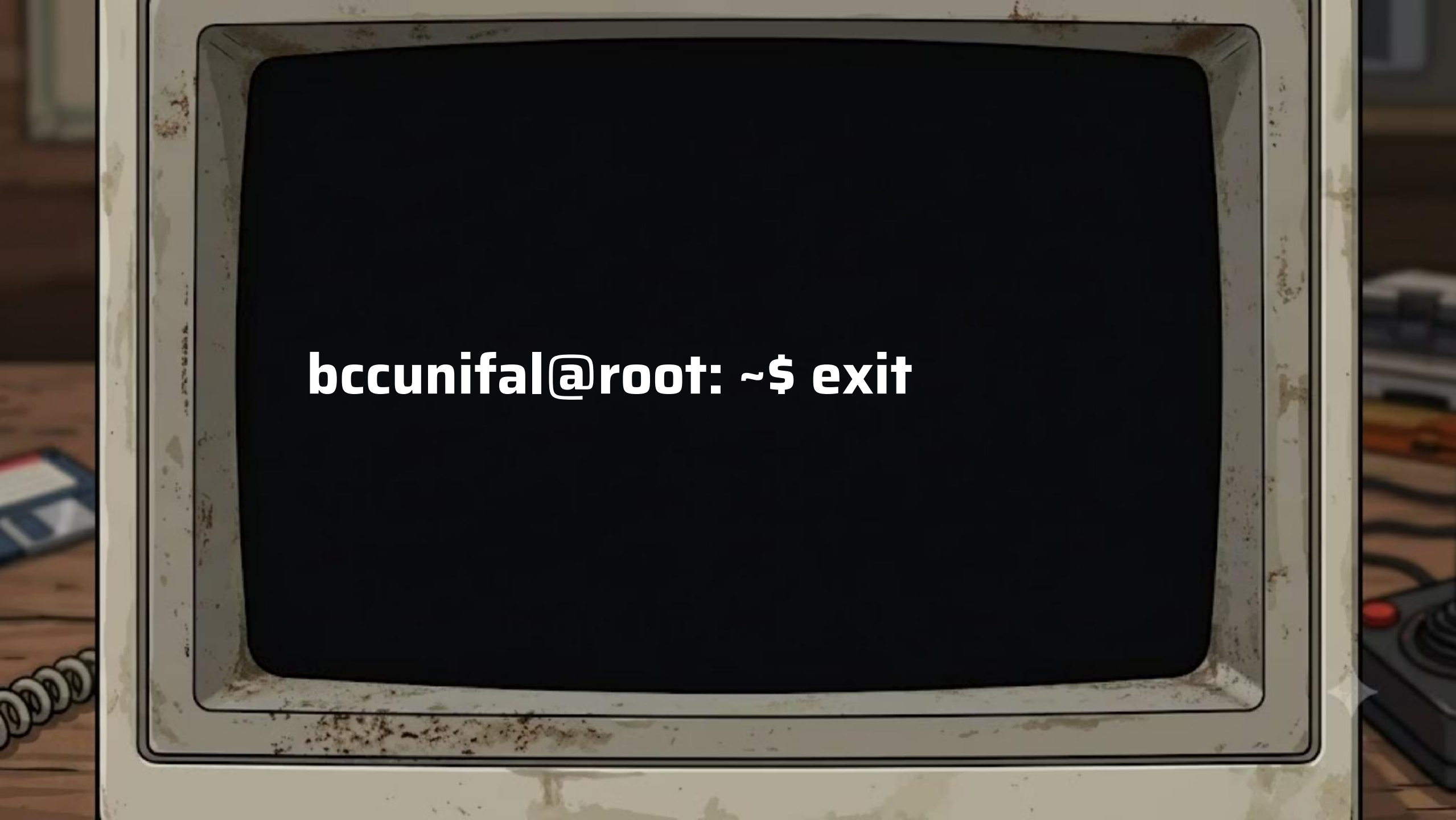
Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin white border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the image shows a wooden desk with a coiled cable on the left and a mouse on the right.

bccunifal@root: ~\$ exit